# VELLORE INSTITUTE OF TECHNOLOGY

[School / Department name]

Operating Systems – Course Project

**REVIEW 2**

Implementation Details

# MediQueue: An AI-Assisted Emergency Department Queue Built on Priority Scheduling and Process Synchronization

Submitted by


| Name | Register Number |
|---|---|
| Aahan Anand | 24BCE2564 |
| Ayush Anupam Sahu | 24BAI0123 |
| Ankit Dhupal | 24BCB0142 |


Under the guidance of

**[Faculty Name]**

GitHub: https://github.com/anandaahan7-prog/mediqueue

Live demo: https://mediqueue-eight-tan.vercel.app

# 1. Project Overview

MediQueue is a web application for the emergency (casualty) department of a hospital. The nurse at the triage desk enters the patient's vital signs and gets an AI suggestion for the triage level. When a doctor becomes free and presses "Call next patient", a priority scheduler decides which patient goes in. Observation beds are handed out through a counting semaphore, and the waiting room screen shows token numbers with an estimated waiting time.

From the OS syllabus we implemented two concepts: CPU scheduling (priority scheduling with aging and limited preemption, with doctors acting as processors) and process synchronization (critical sections and a counting semaphore). This document explains how the system was implemented, the tools used and how it is configured.

# 2. Technology Stack

Table 1: Technology stack

| Layer | Technology | Version | Used for |
|---|---|---|---|
| Backend | Python | 3.12 | Main programming language |
| Web framework | Flask | 3.1.3 | Routes, sessions, templates, JSON API |
| Database | SQLite (WAL mode) | 3.x (built in) | Patients, visits, beds, users, audit log |
| Machine learning | scikit-learn | 1.8.0 | Random forest and gradient boosting models |
| Numerical | NumPy, SciPy | 2.4.4, 1.17.1 | Arrays, dataset generation |
| Model storage | joblib | 1.5.3 | Saving and loading trained models |
| Password hashing | Werkzeug security | (with Flask) | Salted password hashes |
| Frontend | HTML5, CSS3, JavaScript | – | User interface, live refresh every 5 s |
| Templates | Jinja2 | (with Flask) | Server side HTML pages |
| Charts | Hand written SVG (charts.js) | – | Graphs on the admin page |
| Testing | unittest, threading | Python standard library | Functional and concurrency tests |
| Version control | Git, GitHub | – | Source code hosting |
| Deployment | Vercel (Python serverless) | – | Live demo |
| Report graphs | Matplotlib, Playwright | – | Figures and screenshots (not part of the app) |

## 3. Programming Languages Used

Table 2: Languages used in the project

| Language | Where it is used | Approx. lines |
|---|---|---|
| Python | Backend, scheduler, synchronization, AI models, simulation, tests | 2040 |
| SQL (SQLite) | Schema, constraints, indexes, view, all queries | 95 (schema) + inline queries |
| JavaScript | Live refresh of all screens, triage preview, charts | 520 |
| HTML (Jinja2) | 7 page templates | 400 |
| CSS | Styling, light and dark theme, responsive layout | 790 |

We did not use any frontend framework like React. Plain JavaScript was enough because every screen just polls one JSON endpoint and redraws itself. SQL queries are written by hand instead of using an ORM, so that every query, transaction and lock is visible in the code.

## 4. System Architecture

The application follows a three tier architecture (Fig 1). The browser screens request /api/state every 5 seconds. Flask handles each request in its own thread, so the dispatcher and the bed module can be called by several threads at the same time, which is why they are protected with a lock. The trained models are stored as .joblib files and loaded once when the server starts.

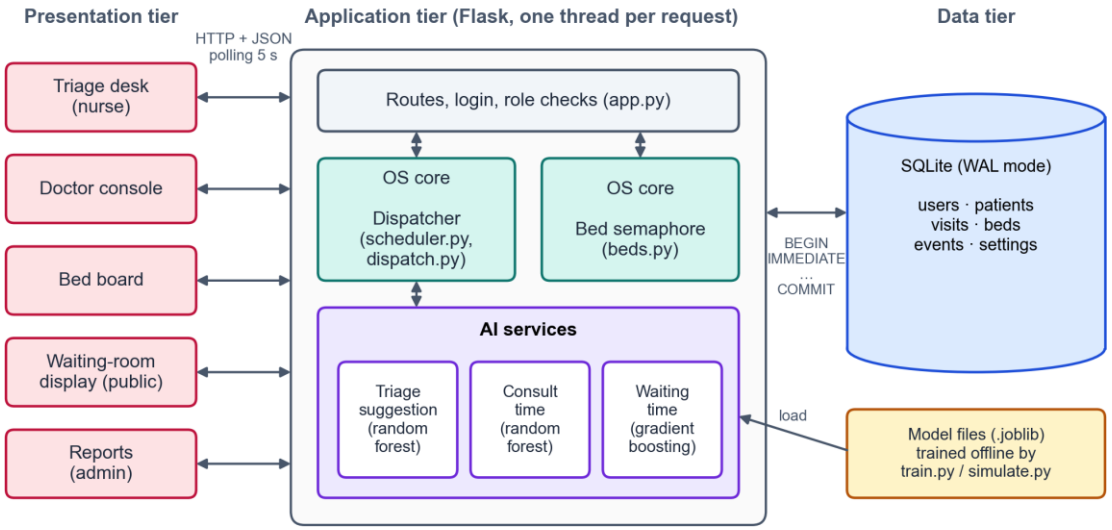



Fig 1: Architecture of MediQueue

## 5. Project Structure

```
mediqueue/
|-- app.py               Flask routes, login, live state API
|-- scheduler.py         scheduling logic (aging, order, preemption, ETA)
|-- dispatch.py          "call next" + ESI 1 preemption (critical section)
|-- beds.py              observation beds as a counting semaphore
|-- db.py                database connection, IST clock, helper functions
|-- schema.sql           tables, constraints, indexes, view
|-- seed.py              demo staff, beds and patients
|-- simulate.py          simulation of 12 hour shifts, policy comparison
|-- race_test.py         concurrency stress test (with and without lock)
|-- ai/
|    |-- dataset.py      synthetic triage dataset generator
|    |-- train.py        trains and evaluates the two random forests
|    |-- triage_model.py runtime: triage suggestion, explanation, time
|    |-- wait_model.py   gradient boosting waiting-time correction
|-- models/              trained .joblib files and result .json files
|-- templates/           login, triage, doctor, beds, board, admin pages
|-- static/              app.js, charts.js, style.css, app.css
|-- tests/               test_system.py (14 tests)
|-- docs/                project report PDFs
|-- requirements.txt, .python-version, .vercelignore, README.md
```

## 6. Database Implementation

The database is SQLite. It needs no separate server, which keeps the project easy to run on any laptop. Fig 2 shows the schema and Table 3 lists the tables.

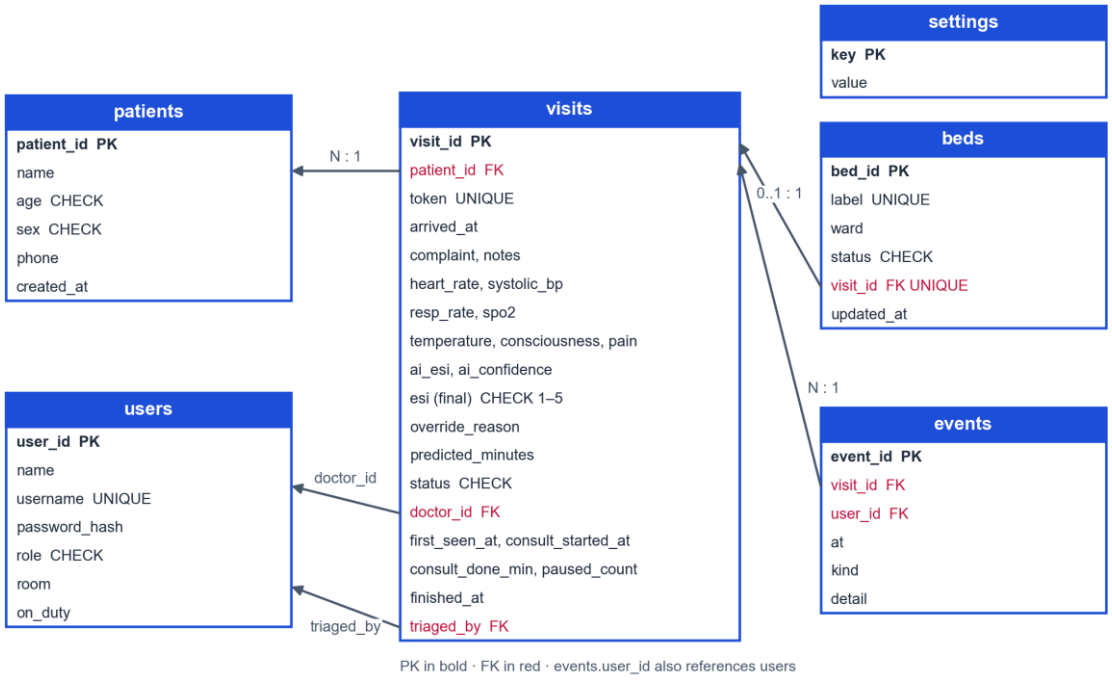



Fig 2: Database schema

Table 3: Database tables

| Table | Purpose | Important constraints |
|---|---|---|

| | | |
|---|---|---|
| users | Nurses, doctors and admin | username UNIQUE, role CHECK, on_duty CHECK |
| patients | Basic patient details | age CHECK 0–120, sex CHECK |
| visits | One row per emergency visit (acts like the PCB) | token UNIQUE, esi CHECK 1–5, status CHECK, FKs |
| beds | 8 observation beds | visit_id UNIQUE (one bed per patient), status CHECK |
| events | Audit log of every decision | FKs to visits and users |
| settings | Configurable values (aging interval) | key PRIMARY KEY |

Indexes are created on visits(status, esi, arrived_at), visits(doctor_id, status) and events(visit_id, at), which are the columns used by the queue and the doctor console. A view v_queue joins visits with patients for the live screens.

# 7. Implementation of Modules

### 7.1 Scheduler (scheduler.py)

The scheduling rules are written as plain functions that work on dictionaries, so the same code is used by the website and by the simulator. The effective level of a waiting patient is calculated with aging that stops at level 3:

```
AGING_MINUTES = 30
AGING_CAP = 3

def effective_level(esi, waited, aging_minutes=AGING_MINUTES):
    if esi <= 2:
        return esi
    return max(AGING_CAP, esi - int(waited // aging_minutes))

def queue_key(visit, now, aging_minutes=AGING_MINUTES):
    waited = now - visit["arrived"]
    level = effective_level(visit["esi"], waited, aging_minutes)
    # smaller key = called first
    return (level, visit["arrived"], visit["id"])
```

For ESI 1 preemption, preemption_target() picks the doctor whose current patient is the least urgent (ESI 3 or higher). If every doctor is with an ESI 1 or 2 patient, nobody is disturbed. The same file also has estimate_waits(), which replays the queue forward with the predicted consultation times to get each patient's waiting time.

### 7.2 Critical section for "Call next patient" (dispatch.py)

Two doctors can press the button at the same moment. To stop them from getting the same patient, the read and the update are done inside one transaction. BEGIN

IMMEDIATE takes the SQLite write lock (like wait on a mutex) and COMMIT releases it (like signal).

```
def call_next(conn, doctor_id):
    conn.execute("BEGIN IMMEDIATE")        # enter critical section
    try:
        rows = conn.execute(
            "SELECT * FROM visits WHERE status = 'waiting'").fetchall()
        jobs = [as_job(r) for r in rows]
        nxt = pick_next(jobs, now_minutes(), "mediqueue", aging)
        cur = conn.execute(
            "UPDATE visits SET status = 'in_consult', doctor_id = ? "
            "WHERE visit_id = ? AND status = 'waiting'",
            (doctor_id, nxt["id"]))
        if cur.rowcount != 1:
            raise DispatchError("Patient was already taken")
        conn.execute("COMMIT")             # leave critical section
    except Exception:
        conn.execute("ROLLBACK")
        raise
```

We used the database lock instead of Python's threading.Lock because a Python lock only works inside one process, while the database lock also works if the server runs several worker processes.

### 7.3 Observation beds as a counting semaphore (beds.py)

The number of free beds is the semaphore value. wait_bed() is called when a doctor admits a patient: it takes a free bed, or puts the patient in the boarding queue (blocked). signal_bed() is called when housekeeping marks a bed as clean: the bed goes straight to the most urgent boarding patient, otherwise it becomes free.

```
def wait_bed(conn, visit_id):     # P(beds) - caller already holds the lock
    bed = conn.execute(
        "SELECT bed_id, label FROM beds WHERE status = 'free' "
        "ORDER BY bed_id LIMIT 1").fetchone()
    if bed is None:                                  # no bed: block
        conn.execute("UPDATE visits SET status = 'boarding' "
                     "WHERE visit_id = ?", (visit_id,))
        return None
    conn.execute("UPDATE beds SET status = 'occupied', visit_id = ? "
                 "WHERE bed_id = ? AND status = 'free'",
                 (visit_id, bed["bed_id"]))
    conn.execute("UPDATE visits SET status = 'admitted' "
                 "WHERE visit_id = ?", (visit_id,))
    return bed["label"]
```

### 7.4 AI triage suggestion (ai/triage_model.py)

The random forest gives a probability for each ESI level. Since under-triage is the dangerous mistake, we move one level up when the model gives the more urgent level at least 30 % probability, and some red flags directly give ESI 1. The reasons shown to the

nurse are found by setting each abnormal value back to normal and checking how much the urgency drops (all variants are sent to the model in one batch, which brought the response time down from about 52 ms to about 16 ms).

```
SAFETY_MARGIN = 0.30

def apply_safety(probs):
    level = int(np.argmax(probs)) + 1
    if level > 1 and probs[level - 2] >= SAFETY_MARGIN:
        level -= 1
    return level

def suggest(vitals):
    probs = probabilities(vitals)
    level = apply_safety(probs)
    if red_flags(vitals):     # e.g. SpO2 < 85, SBP < 80, AVPU >= P
        level = 1
    return {"esi": level, "probs": probs,
            "minutes": predict_minutes(vitals, level),
            "reasons": explain(vitals, probs)}
```

### 7.5 Consultation time and waiting time models

ai/train.py trains a second random forest for the consultation time (it gets the final ESI level as an extra input). ai/wait_model.py has a gradient boosting model that takes the scheduler's own estimate together with the queue state (patients ahead, doctors busy, arrivals in the last hour, hour of the day) and predicts the actual wait. It is trained in simulate.py on 80 simulated shifts and tested on 25 other shifts.

### 7.6 Web application (app.py)

Table 4: Main routes

| Route | Method | Who | What it does |
|---|---|---|---|
| /login, /logout | GET, POST | All | Session login with hashed passwords |
| /triage | GET, POST | Nurse | Triage form, issue token, waiting queue |
| /api/triage/preview | POST | Nurse | Live AI suggestion while typing |
| /visits/<id>/retriage | POST | Nurse | Update vitals, level can only go up |
| /visits/<id>/left | POST | Nurse | Patient left without being seen |
| /doctor | GET | Doctor | Doctor console |
| /doctor/call-next | POST | Doctor | Dispatcher (critical section) |
| /doctor/preempt | POST | Doctor | Pause current patient for ESI 1 |
| /doctor/discharge, /doctor/admit | POST | Doctor | Finish consultation, admit uses wait_bed() |
| /doctor/duty | POST | Doctor | Start / end shift |
| /beds, /beds/<id>/release, /beds/<id>/clean | GET, POST | Staff | Bed board, signal_bed() on clean |
| /board, /api/board | GET | Public | Waiting room display (no names) |
| /admin, /api/admin/reports | GET | Admin | Reports, simulation and model results |
| /admin/settings | POST | Admin | Change aging interval |

| | | | |
|---|---|---|---|
| /admin/race-test | POST | Admin | Run the concurrency test on a scratch database |
| /api/state | GET | Staff | Queue, doctors, beds, stats, log (polled every 5 s) |

### 7.7 Frontend (templates and static)

Each page is a Jinja2 template that extends base.html (sidebar, top bar, flash messages). static/app.js has one function per page (initTriage, initDoctor, initBeds, initBoard, initAdmin) which fetches /api/state every 5 seconds and redraws the queue, doctor cards, beds and log. The triage form calls /api/triage/preview 250 ms after the nurse stops typing. The waiting room display (/board) does not need a login and shows only tokens, rooms and waiting ranges.

## 8. Configuration Details

### 8.1 Software requirements

```
# requirements.txt
flask==3.1.3
scikit-learn==1.8.0
numpy==2.4.4
scipy==1.17.1
joblib==1.5.3

# .python-version
3.12
```

The versions are pinned because the .joblib model files must be loaded with the same scikit-learn version that created them.

### 8.2 Environment variables

Table 5: Environment variables

| Variable | Default | Purpose |
|---|---|---|
| MEDIQUEUE_DB | mediqueue.db in project folder | Path of the SQLite file (tests use a temporary file) |
| VERCEL | set automatically by Vercel | Database is created in /tmp, results written to /tmp |
| SECRET_KEY | development key | Flask session signing key (must be changed in production) |

## 8.3 Application settings

Table 6: Application settings

| Setting | Value | Where |
|---|---|---|
| Aging interval | 30 minutes (admin can change) | settings table / scheduler.py |
| Aging cap | Level 3 | scheduler.py |
| ESI target times | 0, 10, 30, 60, 120 min | scheduler.py |
| Preemption | Only for ESI 1, only patients ESI 3–5 are paused | scheduler.py, dispatch.py |
| Safety margin | 0.30 | ai/triage_model.py |
| Screen refresh | Every 5 seconds | static/app.js |
| Clock | Fixed IST (UTC + 5:30) | db.py |
| Observation beds | 8 (OBS-1 to OBS-8) | seed.py |
| Demo accounts | priya, arun (nurse); rao, mehta, iyer, khan (doctor); admin – password demo123 | seed.py |

## 8.4 Model hyperparameters

Table 7: Model configuration

| Model | Algorithm | Hyperparameters | Training data |
|---|---|---|---|
| Triage level | RandomForestClassifier | 200 trees, max_depth 12, min_samples_leaf 2, class_weight balanced | 3000 synthetic cases, 5-fold CV |
| Consultation time | RandomForestRegressor | 150 trees, max_depth 10, min_samples_leaf 4 | Same 3000 cases + final ESI |
| Waiting time | GradientBoostingRegressor | 300 trees, depth 3, learning rate 0.05, subsample 0.8, absolute error loss | 6342 patients from 80 simulated shifts |

## 8.5 Database configuration

- PRAGMA journal_mode = WAL, so the screens that only read data do not block the writer.
- PRAGMA busy_timeout = 15000, so a request waiting for the lock waits up to 15 seconds instead of failing.
- PRAGMA foreign_keys = ON for referential integrity.
- Connections are opened with isolation_level=None so that BEGIN IMMEDIATE and COMMIT are written explicitly in the code.

## 8.6 Deployment configuration

- Vercel detects the Flask app in app.py automatically and runs it as a Python serverless function.

- .vercelignore leaves out tests, docs and the training scripts to keep the deployment small.
- Since Vercel only allows writing to /tmp, ensure_database() in app.py creates and seeds the demo database there when a server instance starts.
- Every response has an X-MediQueue-Instance header that shows which server instance answered.

## 9. Steps to Run the Project

```
git clone https://github.com/anandaahan7-prog/mediqueue.git
cd mediqueue
pip install -r requirements.txt
python seed.py                # creates mediqueue.db with demo data
python app.py                 # open http://localhost:5000

python -m unittest tests.test_system -v    # 14 tests
python race_test.py                        # concurrency test
python -m ai.train            # retrain triage models (optional)
python simulate.py            # policy comparison (optional, ~30 s)
```

## 10. Testing

tests/test_system.py has 14 tests. Each test starts from a fresh demo database and they all pass in about 25 seconds.

Table 8: Test cases

| Test | Expected result | Result |
|---|---|---|
| Login with wrong password | Error, no session | Pass |
| Nurse opens admin page | Access denied | Pass |
| Heart attack vitals | ESI 1 or 2 | Pass |
| SpO2 82 %, responds to pain | Red flag, ESI 1 | Pass |
| Override without reason | Rejected | Pass |
| Doctor calls next | Gets the first patient in the queue | Pass |
| Call next while already busy | Refused | Pass |
| ESI 5 patient waiting 78 min | Aged to level 3, above newer ESI 3 | Pass |
| 4 doctors call at the same time | 4 different patients | Pass |
| ESI 1 arrives, all doctors busy | ESI 3–5 consultation paused | Pass |
| Worse vitals on re-triage | Level goes up | Pass |
| Admit when no bed is free | Boarding, then bed given on clean | Pass |
| No bed has two patients | No duplicates | Pass |
| Waiting room API | No patient names | Pass |

race_test.py runs 6 threads that press "call next" together and 8 threads that try to admit patients when only 3 beds are free, 30 times each. Without the lock we got 45 patients

assigned to two doctors and 50 beds given to more than one patient. With the critical section there were no conflicts. The average time per request went up from about 22 ms to 35 ms because the requests now wait for each other.

## 11. Source Code and Deployment

Table 9: Links

| Item | Link |
|---|---|
| GitHub repository (public) | https://github.com/anandaahan7-prog/mediqueue |
| Live deployment (Vercel) | https://mediqueue-eight-tan.vercel.app |
| Login | e.g. priya / demo123 (nurse), iyer / demo123 (doctor), admin / demo123 |

The repository is public so that it can be accessed for review. Note about the live demo: on Vercel every server instance keeps its own copy of the SQLite database in /tmp. With one person using it in one browser tab everything works normally. If many requests come at the same moment, Vercel can start a second instance which does not share the same data. For the full demo with nurse, doctor and waiting room screens open together, we run the project locally (python app.py). Moving the database to a shared hosted database would remove this limitation.

Screenshots of the output will be submitted in Review 3.